Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 – «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR'S GRADUATE THESIS)
Field of Study
09.03.01 – «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»

Тема / Topic	**Система уточняющих типов для языка Dart / Refinement type system for Dart**

Работу выполнил / Thesis is executed by	**Чистяков Климентий Павлович / Chistyakov Klimentii Pavlovich**	подпись / signature
Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor	**Николай Кудасов Дмитриевич / Nikolai Kudasov Dmitrievich**	подпись / signature
Консультанты /		

Consultants		подпись / signature

Иннополис, Innopolis, 2026

Contents

1 Introduction **8**

1.1 Background & Relevance 8

1.2 Problem Statement 8

1.3 Research Hypothesis 9

1.4 Objectives 10

1.5 Limitations & Scope 10

1.6 Dart Analyzer Capabilities (Baseline) 11

1.7 Evaluation Criteria 11

1.8 Evaluation Methodology 12

1.8.1 Corpus Selection 12

1.8.2 Baseline Analysis 13

1.8.3 Extended Analysis 13

2 Literature Review **14**

2.1 Refinement Types 14

2.2 Liquid Types and Refinement Type Inference 14

2.3 Refinement Types in Other Languages 15

2.4 SMT Solvers and Z3 16

3 Methodology **17**

4 Implementation **18**

5 Evaluation and Discussion 19

6 Conclusion 20

Bibliography cited 21

List of Tables

3.1 This is the title I want to appear in the List of Tables 17

List of Figures

Abstract

My abstract starts from here.

Chapter 1

Introduction

1.1 Background & Relevance

Index out of bounds errors are a well known reason for critical bugs appearing in modern software, mainly occurring in places where the checks for proper indexing are either insufficient or lacking entirely. The most notable example of these errors is access to an element of array that doesn't exist at the index provided.

Since a lot of programming languages rely on the array as a commonly used data structure, it is easy to encounter such an error in any part of the developed software. The examples include but are not limited to color conversions [1] [2], pixel values [3] or handling of empty lists [4].

Unfortunately, due to the nature of how array accesses happen, these errors are quite difficult to notice until they reach runtime or sometimes even production environments completely unnoticed.

1.2 Problem Statement

Dart programming language is prone to index out of bounds errors. The current type system, though increasing robustness [5], does not currently support

collection-length constraints. This makes developers rely on assumptions and runtime testing.

This thesis attempts to resolve this issue by introducing a refinement type system to statically confirm safety of array access operations while adding little annotation burden to the developers.

To evaluate the practical applicability of the proposed approach, the implementation will be tested on a corpus of publicly available Dart packages obtained from pub.dev (Dart package store). This evaluation will investigate how effectively the proposed refinement type system can verify list indexing operations in existing software projects.

1.3 Research Hypothesis

This thesis hypothesises that the introduction of refinement types into the Dart static analysis workflow will improve safety of list indexing operations, compared to the standard Dart analyzer, while maintaining a reasonable annotation burden and analysis overhead.
The hypothesis will be evaluated using the following critera:

- At least $X\%$ of list indexing operations in the selected package corpus will have their safety verified by the proposed implementation
- The annotation burden will not exceed Y refinement annotations per list indexing operation
- The additional analysis time will not exceed $Z\%$ of the baseline analysis time

An indexing operation will be considered either proven safe, proven unsafe or unverifiable. An unverifiable operation doesn't necessarily repersent a runtime

error, but rather a case where it is impossible to statically determine safety of this operation with all information available.

1.4 Objectives

We aim to achieve the following goals:

- An implementation of an analyzer for a custom predicate language
- An implementation of a refinement type checker and type inference algorithm with usage of Z3 constraints solver
- Integration of said checker into Dart analysis workflow [6]
- Evaluation of the effectivness of the implementation on a package corpus from pub.dev

1.5 Limitations & Scope

The implementation will cover the following subset of Dart programming language and will allow users to write the following refinements:

- Refinement type definitions that will allow users to extend integers, booleans and lists with refinements
- Definition of variables with refinement types
- Definition of recursive/non-recursive functions
- Usage of $length(xs)$ function in refinements for List type and its refinement extensions

1.6 Dart Analyzer Capabilities (Baseline)

The Dart analyzer already provides a number of static analysis features, including but not limited to type checking, type inference, null safety and control flow analysis [5]. These features allow the analyzer to identify and report numerous programming errors before runtime.

However, the existing type system doesn't generally express or verify relationships between collection lengths and their indecies.

For example, given a list xs and an integer index value index, the analyzer can confirm that the expression $\mathrm{xs}[\mathrm{index}]$ is safe. However it cannot verify that the following holds:

$$0 \leq i < \mathrm{xs.length}$$

As a result, an expression that is valid per static analysis will throw $\mathrm{RangeError}$ at runtime.

The proposed implementation aims to extend these existing capabilities by introducing refinement types and constraint solving, allowing collection length and indexing operations properties to be verified statically

1.7 Evaluation Criteria

The implementation will be evaluated using three primary metrics:

- **Verification rate:** The proportion of eligble list indexing operations whose safety is successfully verified by the checker
- **Annotation burden:** The ratio between the number of refinement annotations added and the total number of eligble list indexing operations

- **Analysis overhead:** The relative increase in analysis time when using the proposed implementation, compared to the standard Dart analyzer

The following thresholds will be used to evaluate the hypothesis:

- Verification rate of at least X%
- Annotation burden no bigger than Y
- Analysis overhead not exceeding Z% of the baseline analysis time

Additionaly, the number of indexing operations proven unsafe/unverifiable will be recorded to provide more insights onto the effectivness and limitations of the implementation

1.8 Evaluation Methodology

The effectivness of the implementation will be evaluated using the corpus of packages from Dart package store pub.dev.

1.8.1 Corpus Selection

The initial corpus will consist of 30 packages, selected from the 100 most popular packages on pub.dev at the time of corpus construction.

Packages will be selected randomly and must satisfy the following conditions:

- The package must have a stable release.
- The package must be compatible with the Dart SDK version used in the experiment.
- The package must provide Dart source code.

- The package must contain at least one list indexing in its production source code.

The analysis will focus on production source files, located in `lib/` directory. Generated sources will be excluded.

1.8.2 Baseline Analysis

Each selected package will first be analyzed with standard Dart analyzer without the proposed refinement checker.

We will record the diagnostics produced by the analyzer, analysis time and the number of eligble indexing operations.

This experiment establishes capabilities of the existing Dart analysis workflows and provides a baseline for evaluating verification capabilities of the proposed implementation.

1.8.3 Extended Analysis

The same packages will be analyzed using the proposed type checker, with necessary refinement annotations added.

For each of the eligble indexing operations, the implementation will record the following:

- Proven safe.
- Proven unsafe.
- Unverifiable.

The number of introduced annotations and analysis time will also be recorded. The results will be compared against the baseline analysis using evaluation criteria defined in 1.7

Chapter 2

Literature Review

2.1 Refinement Types

Refinement types allow the developer to extend a language's type system with predicates [7]. For example, the user can constraint the regular integer type with the following refinement to get the type that represents only natural numbers.

```
1  @Refinement(identifier: 'x', predicate: 'x >= 0')
2  typedef nat = int;
```

With such refinements the developer can easily avoid index out of bounds errors by, for example, defining the following refinement:

```
1  @Refinement(parameters: ['list', 'index'], predicate: 'index
       <= length(list)')
2  T get(List<T> list, int index) => list[index];
```

The type checker will ensure that the index provided to the function is smaller than the length of the list, and if not, will report this operation as unsafe.

2.2 Liquid Types and Refinement Type Inference

Annotating each variable, each type with refinement information can get overwhelming and consume more time than the annotations will reduce in error

finding. To address this issue we can implement refinement type inference using *Logically Qualified Data Types* also known as *Liquid Types* system explored by *Rondon et al.*[8] which will provide the developer with strength of refinement types without additional unnecessary effort. The inference process involves the following steps:

1. Hindley-Milner type inference [9]
2. Constraint generation
3. Constraint solving using an SMT solver [10]

Assume a function *safeHead*, where we want to check safety of *xs[0]* operation:

```
1    int safeHead(List<int> xs) => xs.length > 0 ? xs[0] : 0;
```

Without type inference we would need to manually specify length property of *xs* as a refinement, however liquid types can infer the constraint from the program itself.

2.3 Refinement Types in Other Languages

Similar implementations exist for other programming languages. The most similar one to my implementation is a refinement type system for TypeScript [11] by *Vekris et al*. Their work demonstrates an integration of a refinement type system into an imperative object-oriented language while retaining support for higher-order functions.

One of the main problems the implementation faces is treatment of mutable variables. Since the value of a variable can change during program execution, refinement predicates can become invalid after reassignment. To address the issue the authors translate input programs into Static Single Assignment form, where each variable is assigned only once. This allows their checker to reason about

individual variable versions and retain the information about conditions under which they were assigned.
The implementation also uses the Liquid Types [8] system which, on par with TypeScript being similar to Dart on many accounts [5], makes this implementation a good reference point for my project.
Other notable examples of such system implementations are [12, Liquid Haskell], [13, Refinement Types For Ruby], and [14, Flux: Liquid Types for Rust]

2.4 SMT Solvers and Z3

Satisfiability Modulo Theories (SMT) is a problem that asks whether a first-order logic formula is satisfiable with respect to a background theory [15]. It extends Boolean Satisfiability (SAT) problem by adding reasoning such as integers, arrays, bit-verctos and uninterpreted functions. Since SMT solvers are able of reasoning about relationships between values and operations within theories, they become particularly useful in software verification, where program properties involve arithmetic constraints, equality connections and data structures. de Moura and Bjørner present Z3 [10], an SMT solver developed at Microsoft Research and designed specifically for applications in software verification. In context of refinement type checking, an SMT solver can serve as the reasoning engine, responsible for verifying refinement predicates and generating constraints, making Z3 suitable for proposed refinement type checker for Dart. Using Z3 allows to delegate logical reasoning to Z3 and focus on the implementation on extracting extracting properties and interpreting verification results.

Chapter 3

Methodology

Referencing other chapters 2, 3, 4, 5 and 6

TABLE 3.1
Simulation Parameters

A	B
Parameter	**Value**
Number of vehicles	$\mid \mathcal{V} \mid$
Number of RSUs	$\mid \mathcal{U} \mid$
RSU coverage radius	150 m
V2V communication radius	30 m
Smart vehicle antenna height	1.5 m
RSU antenna height	25 m
Smart vehicle maximum speed	v_{max} m/s
Smart vehicle minimum speed	v_{min} m/s
Common smart vehicle cache capacities	$[50, 100, 150, 200, 250]$ mb
Common RSU cache capacities	$[5000, 1000, 1500, 2000, 2500]$ mb
Common backhaul rates	$[75, 100, 150]$ mb/s

...

Chapter 4

Implementation

...

Chapter 5

Evaluation and Discussion

...

Chapter 6

Conclusion

...

Bibliography cited

[1] bazinac, *RangeError (index): Index out of range: Index should be less than 345600: 10*, https://github.com/flutter/flutter/issues/89584, Sep. 2021. Accessed: Nov. 1, 2026.

[2] *Changelog — widget-recoding-plugin (version 1.0.4)*, https://github.com/abdulhadinaeem/widget-recoding-plugin/blob/1.0.4/CHANGELOG.md. Accessed: Nov. 1, 2026.

[3] volgin, *Index out of bounds when scrolling to the very end*, https://github.com/fluttercommunity/flutter-draggable-scrollbar/issues/8, Dec. 2018. Accessed: Nov. 1, 2026.

[4] NgoTruongThanh, *[syncfusion_flutter_datagrid] Error RangeError (index): Index out of range: No indices are valid: 0*, https://github.com/syncfusion/flutter-widgets/issues/1856, May 2024. Accessed: Nov. 1, 2026.

[5] *Dart-lang/language*, https://github.com/dart-lang/language, Sep. 2026. Accessed: Sep. 28, 2026.

[6] *Analyzer plugins*, https://dart.dev/tools/analyzer-plugins. Accessed: Sep. 28, 2026.

[7] R. Jhala and N. Vazou, *Refinement Types: A Tutorial*, Oct. 2020. DOI: 10.48550/arXiv.2010.07763 arXiv: 2010.07763 [cs.PL]. Accessed: Oct. 4, 2026.

[8] P. M. Rondon, M. Kawaguci, and R. Jhala, "Liquid types," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '08, New York, NY, USA: Association for Computing Machinery, Jun. 2008, pp. 159–169, ISBN: 978-1-59593-860-2. DOI: 10.1145/1375581.1375602 Accessed: Oct. 4, 2026.

[9] L. Damas and R. Milner, "Principal Type-Schemes for Functional Programs,"

[10] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and J. Rehof, Eds., Berlin, Heidelberg: Springer, 2008, pp. 337–340, ISBN: 978-3-540-78800-3. DOI: 10.1007/978-3-540-78800-3_24

[11] P. Vekris, B. Cosman, and R. Jhala, "Refinement types for TypeScript," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '16, New York, NY, USA: Association for Computing Machinery, Jun. 2016, pp. 310–325, ISBN: 978-1-4503-4261-2. DOI: 10.1145/2908080.2908110 Accessed: Oct. 4, 2026.

[12] N. Vazou, E. L. Seidel, and R. Jhala, "LiquidHaskell: Experience with refinement types in the real world," in *Proceedings of the 2014 ACM SIGPLAN Symposium on Haskell*, ser. Haskell '14, New York, NY, USA: Association for Computing Machinery, Sep. 2014, pp. 39–51, ISBN: 978-1-4503-3041-1. DOI: 10.1145/2633357.2633366 Accessed: Oct. 4, 2026.

[13] M. Kazerounian, N. Vazou, A. Bourgerie, J. S. Foster, and E. Torlak, "Refinement Types for Ruby," in *Verification, Model Checking, and Abstract Interpretation*, I. Dillig and J. Palsberg, Eds., Cham: Springer International Publishing, 2018, pp. 269–290, ISBN: 978-3-319-73721-8. DOI: 10.1007/978-3-319-73721-8_13

[14] N. Lehmann, A. T. Geller, N. Vazou, and R. Jhala, "Flux: Liquid Types for Rust," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, 169:1533–169:1557, Jun. 2023. DOI: 10.1145/3591283 Accessed: Oct. 4, 2026.

[15] C. Barrett, R. Sebastiani, S. A. Seshia, and C. Tinelli, "Chapter 33. Satisfiability Modulo Theories," in *Handbook of Satisfiability*, ser. Frontiers in Artificial Intelligence and Applications, A. Biere, M. Heule, H. Van Maaren, and T. Walsh, Eds., IOS Press, Feb. 2021, ISBN: 978-1-64368-160-3 978-1-64368-161-0. DOI: 10.3233/FAIA201017 Accessed: Oct. 30, 2026.